



Anwendungsarchitektur

Tim Schmitz
16.05.2026

Inhalt

Übersicht	3
Hauptklasse	4
Eigenschaftenklasse	5
Datenbank-Eigenschaftenklasse	6
Verbindungsklasse	7
Domänen-Schema	7
Tabellen-Schema	8
Anwendungssteuerung	8

Anwendungsarchitektur

Jede Anwendung benötigt eine Architektur, also eine Struktur und einen Plan, indem das innere Gerüst der Anwendung organisiert ist. Die Architektur einer Anwendung sollte so stabil und nachhaltig sein, dass sie sich langfristig nicht ändert. Was sich in der Regel ändert, sind Funktionen und Bedingungen, die von außerhalb, d.h. von der Umgebung und dem Systemkontext, stammen.

Übersicht

Die folgende Übersicht zeigt eine einfache Anwendungsarchitektur, deren Klassengerüst aus sieben Klassen und drei Paketen besteht.

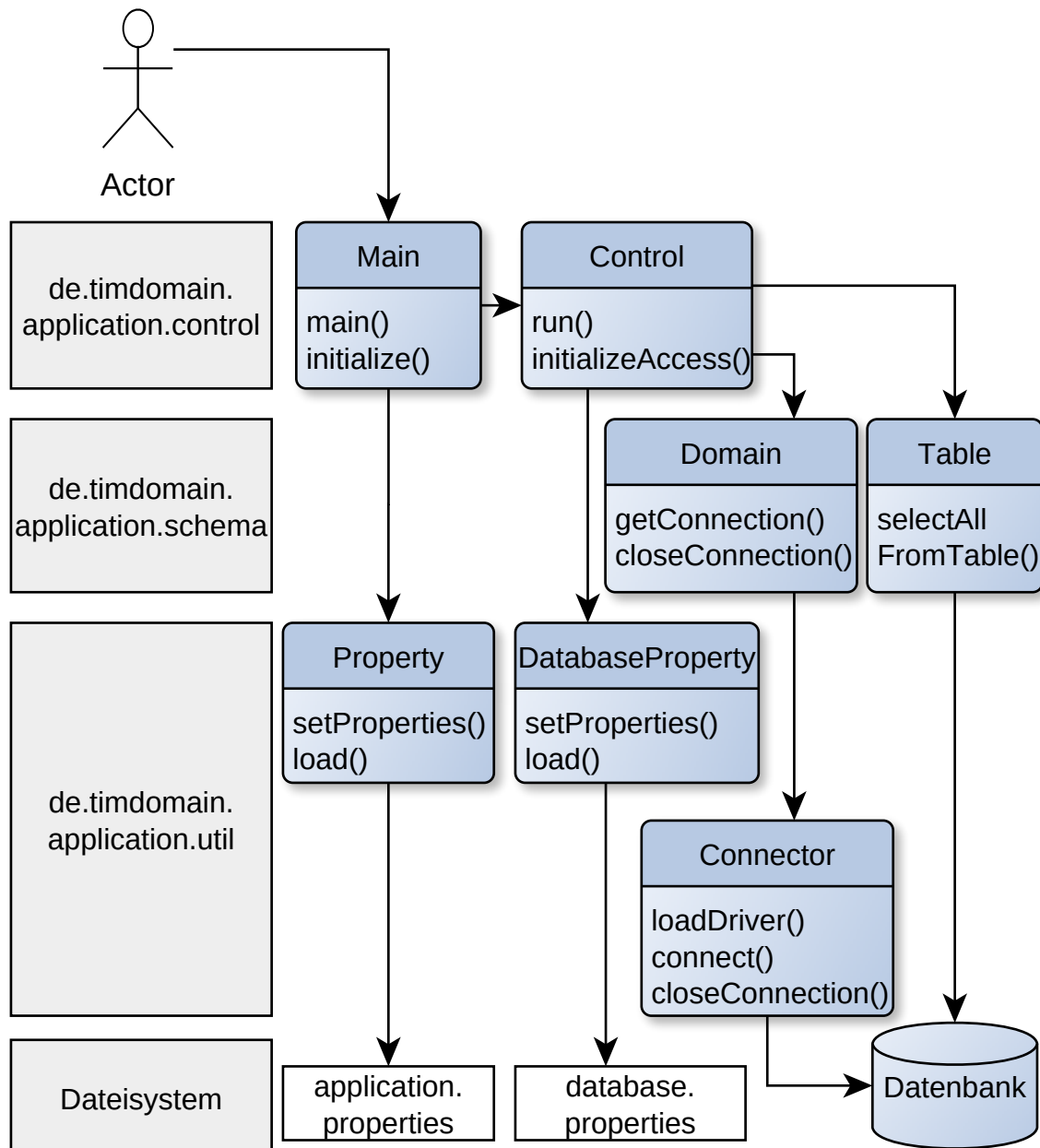


Abbildung 1: Anwendungsarchitektur

Der Benutzer (Akteur) startet die Anwendung, indem er über das Exekutionsmodul (application.bat) die Hauptklasse der Anwendung Main ruft. Die Hauptmethode main() der Hauptklasse initialisiert

zunächst den Systemkontext. Dazu benutzt sie die Klasse Property, welche die Datei mit den Anwendungseigenschaften, die in der Datei application.properties gespeichert sind, lokalisiert und anschließend diese lädt. Hiernach kann die Hauptklasse verschiedene System-Properties wie den Rechnernamen und den davon abhängigen Logging-Pfad bestimmen. Anschließend wird der Logger instantiiert und die Steuerung an die eigentlich Anwendungssteuerung, die Klasse Control mit ihrer Hauptmethode run() weitergegeben.

Die Klassen sind in folgenden Paketen organisiert:

- de.timdomain.application.control: dieses Paket beinhaltet Hauptklasse und Anwendungssteuerung
- de.timdomain.application.schema: dieses Paket beinhaltet das Domänen- und Tabellen-Schema
- de.timdomain.application.util: dieses Paket beinhaltet Hilfsklassen wie Connector und Property-Klassen

Eigenständig von der Anwendung, aber mit ihr verbunden sind Komponenten des Datei-Systems wie die Properties oder die Datenbank.

Generell werden folgende Konventionen verwendet:

- Alle Klassen sind in der Regel public
- Alle Methoden sind public und static
- Alle Variablen und Konstanten sind private und static

In begründeten Ausnahmefällen werden auch Instanzmethoden oder der Modifier protected verwendet.

Hauptklasse

Die Klasse Main ist die Hauptklasse der Anwendung. Sie inspiziert die System-Umgebung der Anwendung und legt in Abhängigkeit von der Umgebung fest, wohin die Anwendung gelogget wird. Inhaltlich leistet die Hauptklasse nichts, sondern delegiert die Kontrolle an die Anwendungssteuerung Control.

Die Hauptmethode der Hauptklasse heißt main() und startet die Anwendung. Sie initialisiert den Umgebungskontext, setzt die System-Properties für den Rechnernamen und die Logging-Pfad aufgrund der initialisierten Daten und schreibt das so definierte Logging.

```
public static void main(String[] args) {  
    // Initialisierung  
    initialize();  
  
    // Setzen der System-Properties  
    String computerName = getHostName();  
    String logPath = setLogPath(computerName);  
  
    // Parameter, um den Workflow zu steuern  
    Logger LOG = LogManager.getLogger(MainRun.class.getName());  
    LOG.trace("LogPath: " + logPath);  
  
    // Aufruf der Anwendung  
    Control.run(computerName);  
}
```

Die Initialisierungsmethode heißt initialize(). Sie setzt die Properties der Anwendung und lädt sie. Dann werden die möglichen Computernamen für alle Rechner-Varianten und ihre zugehörigen Logging-Pfade ermittelt sowie eine Fehlermeldung.

```
public static void initialize() {  
    // Application Properties setzen und laden  
    ApplicationProperty.setProperties();  
    ApplicationProperty.load();  
  
    // Computernamen ermitteln  
    META_PC = ApplicationProperty.getMetaPC();  
    TEST_PC = ApplicationProperty.getTestPC();  
  
    // Logging-Pfade ermitteln  
    LOG_PATH_META_PC = ApplicationProperty.getLogPathMeta();  
}
```

```

LOG_PATH_TEST_PC = ApplicationProperty.getLogPathTest();

// Error Message ermitteln
ERROR_HOST = ApplicationProperty.getErrorHost();
}

```

Zusätzlich zu diesen beiden Methoden gibt es noch die Methoden `getHostName()` und `setLogPath()`. Die Methode `getHostName()` ermittelt den Rechnernamen aus dem Umgebungskontext. Dabei benutzt sie die Systemklasse `InetAddress`. Da zu diesem Zeitpunkt noch kein Logger initialisiert wurde, muss eine Ausnahmebehandlung durch die System-Error-Message erfolgen.

```

public static String getHostName() {
    String hostName = "";

    // Den Rechnernamen aus der Umgebung holen
    try {
        hostName = InetAddress.getLocalHost().getHostName();
    } catch (IOException ioException) {
        System.err.println(ioException.getMessage());
    }
    return hostName;
}

```

Die Methode `setLogPath()` setzt den Logging Path abhängig vom Rechnernamen. Dabei wird der Computernamen als Parameter übergeben. In Abhängigkeit vom benötigt wird. Auch hier muss eine Fehlermeldung noch mit der System-Error-Message erfolgen.

```

public static String setLogPath(String computerName) {
    String logPath = "";

    // Abfrage des Rechnernamens und setzen des Logging Pfads
    if (TIM_PC.equals(computerName)) {
        logPath = LOG_PATH_TIM_PC;
    } else if (META_PC.equals(computerName)) {
        logPath = LOG_PATH_META_PC;
    } else if (TEST_PC.equals(computerName)) {
        logPath = LOG_PATH_TEST_PC;
    } else {
        System.err.println(ERROR_HOST);
    }

    // Setzen des System Logging-Pfades
    System.setProperty("log.path", logPath);
    return logPath;
}

```

Eigenschaftenklasse

Die Klasse `ApplicationProperty` ist die Property-Klasse der Anwendung. Sie lädt die Properties, setzt die Keys und ermittelt für jeden Key das entsprechende Property, d.h. den Eigenschaftswert. Dazu liest die Daten aus der Eigenschaftendatei `application.properties` aus. Diese Daten sind in Key-Value-Formaten organisiert.

Die Datei `application.properties` hat folgendes Aussehen:

```

tim.pc = Tim-PC
meta.pc = 16H0054
test.pc = DESKTOP-LERLCJ6
error.host = Kein Computernamen definiert
logpath.tim = C:\\Users\\Tim\\Daten\\Java\\Applications\\application\\logs
logpath.meta = C:\\Users\\TSZ\\Daten\\Java\\Applications\\application\\logs
logpath.test = C:\\Users\\test\\Daten\\Java\\Applications\\application\\logs

```

Die Keys, über die die Properties referenziert werden, bestehen aus zwei oder mehr zusammenhörigen Begriffen, die jeweils durch Punkte miteinander verbunden werden. Die Values bestehen jeweils aus einem String, der technisch als endlich lange Folge von character-Zeichen realisiert wird.

Die Methode `setProperties()` definiert das Property-File samt Pfad, um später darauf zugreifen zu können. Dafür wird das Verzeichnis initialisiert, in dem die Properties-Datei liegt. In diesem Fall ist es das aktuelle Verzeichnis, in dem die Anwendung läuft ("user.dir") und darunter das Unterverzeichnis `properties` - dieselbe Struktur muss deswegen auch in der Ablaufumgebung abgebildet werden.

Anschließend wird die Properties-Datei als File instanziiert sowie ein Properties-Objekt. Im letzten Schritt wird das Setzen der Keys vorgenommen.

```
public static void setProperties() {
    // Statische Variablen für den Zugriff auf die Properties
    String directory = System.getProperty("user.dir") + File.separator + "properties";
    File file = new File(directory, "application.properties");
    Properties properties = new Properties();
    setKeys();
}
```

Die Methode setKey() setzt für jedes Property die Keys, um später einen Value für den jeweiligen Key auszuwerten. Dabei müssen die Keys der Properties-Datei bekannt sein und benutzt werden.

```
private static void setKeys() {
    timPC = "tim.pc";
    metaPC = "meta.pc";
    testPC = "test.pc";
    errorHost = "error.host";
    logPathTim = "logpath.tim";
    logPathMeta = "logpath.meta";
    logPathTest = "logpath.test";
}
```

Die Methode load(), die anschließend von der Hauptklasse der Anwendung aufgerufen wird, erzeugt einen Eingabestrom mit Dateinamen und lädt die Properties des Projekts über die Entsprechende Objektmethode. Da es sich um eine Datei-Operation handelt, muss eine Eingabe-Ausgabe-Ausnahmebehandlung hinzugefügt werden. Damit sind alle Properties der Eigenschaftendatei für die Anwendung zugreifbar.

```
public static void load() {
    try {
        FileInputStream inputStream = new FileInputStream(file);
        properties.load(inputStream);
    } catch (IOException ioException) {
        ioException.getMessage();
    }
}
```

Die Getter der Property-Klasse haben alle dasselbe Aussehen und geben den Value des über den Key referenzierten Properties zurück. Damit werden die Getter zu einfachen Einzeilern.

```
public static String getTimPC() {
    return properties.getProperty(timPC);
}
```

Datenbank-Eigenschaftensklasse

Die Klasse DatabaseProperty ist die Property-Klasse der Datenbank. Sie lädt die Properties für den Datenbank-Zugriff, setzt die Keys und ermittelt für jeden Key das entsprechende Database-Property, d.h. den Eigenschaftswert. Dazu liest die Daten aus der Eigenschaftendatei database.properties aus. Diese Daten sind in Key-Value-Formaten organisiert. Man erkennt schnell, dass die Klasse DatabaseProperty sich genauso verhält wie die Klasse ApplicationProperty und daher dieselben Methoden und Zugriffe verwendet.

Die Datei database.properties hat dabei folgendes Aussehen

```
database.user=tim
database.password.test=TimSql#xxxx
database.password.meta=TimSql#yyyy
database.server.test=DESKTOP-LERLCJ6
database.server.meta=22H0566
database.driver=com.microsoft.sqlserver.jdbc.SQLServerDriver
database.port=jdbc:sqlserver://localhost:1434
```

Die Methode setKey() setzt analog für jedes Property die Keys, um später einen Value für den jeweiligen Key auszuwerten. Dabei müssen die Keys der Properties-Datei bekannt sein und benutzt werden.

```

private static void setKeys() {

    // Login
    databaseUser = "database.user";
    databasePasswordTest = "database.password.test";
    databasePasswordMeta = "database.password.meta";
    databaseServerTest = "database.server.test";
    databaseServerMeta = "database.server.meta";

    // Connector
    databaseDriver = "database.driver";
    databasePort = "database.port";
}

```

Die Methode load() und die Getter dieser Eigenschaftenklasse arbeiten identisch wie in der Klasse ApplicationProperty.

Verbindungsklasse

Die Verbindungsklasse Connector stellt Methoden bereit, mit denen sich die Anwendung mit der Datenbank verbinden und von der Datenbank auch wieder trennen kann. Wie die beiden Property-Klassen ist sie Teil des Util-Pakets, da sie nicht-funktionale Anforderungen realisiert. Dafür müssen zunächst die java.sql-Pakete importiert. Anschließend kann der Datenbank-Treiber mit der Methode loadDriver() geladen werden.

```

public static void loadDriver(String databaseDriver) {
    try {
        Class.forName(databaseDriver);
        LOG.trace("Datenbank-Treiber laden: ");
        LOG.trace(databaseDriver);
    } catch (ClassNotFoundException classNotFoundException) {
        LOG.error("Class Not Found: ");
        LOG.error(classNotFoundException.getMessage());
    }
}

```

Die Methode connect() stellt eine Verbindung zur Datenbank her. Dafür benötigt die Methode den Datenbank-Lokator, den Datenbank-Benutzer und sein Passwort. Der Rückgabewert dieser Methode ist die Verbindung zur Datenbank.

```

public static Connection connect(String databaseURL, String databaseUser, String databasePassword) {
    Connection connection = null;
    try {
        connection = DriverManager.getConnection(databaseURL, databaseUser, databasePassword);
        LOG.trace("Zur Datenbank verbinden: ");
        LOG.trace(databaseURL);
    } catch (SQLException sqlException) {
        LOG.error("SQL-Exception: ");
        LOG.error(sqlException.getMessage());
    }
    return connection;
}

```

Abschließend definiert die Methode closeConnection() die Trennung von der Datenbank. Dafür benötigt die Methode die Verbindung zur Datenbank als Übergabe-Parameter.

```

public static void closeConnection(Connection connection) {
    try {
        if (connection != null) connection.close();
    } catch (SQLException sqlException) {
        LOG.trace("SQL-Exception: ");
        LOG.trace(sqlException.getMessage());
    } finally {
        LOG.trace("Datenbank-Verbindung trennen");
    }
}

```

Domänen-Schema

Das Domänen-Schema repräsentiert in formaler Weise einen Wertebereich, dessen Daten strukturiert sind. In diesem Fall ist die Domäne die spezifische Datenbank des Relationalen Datenbank-Managementsystems SQL Server, hier genannt "CBM".

Da der Connector nicht direkt von der Anwendung angesprochen werden soll, übernimmt diese Aufgabe das Domänen-Schema. Dieses wird seinerseits von der Anwendungssteuerung gerufen. Die Methode `getConnection()` lädt also via des Connectors den Datenbank-Treiber und stellt eben damit auch die Verbindung zur Datenbank her.

```
public static void getConnection(String databaseDriver, String databasePort, String databaseUser,
                                String databasePassword) {
    LOG.trace("Aufbau der SQL Server Connection...");
    Connector.loadDriver(databaseDriver);
    sqlServerConnection = Connector.connect(databasePort + ";databaseName=CBM", databaseUser,
                                           databasePassword);
}
```

Entsprechend ruft die Methode `closeConnection()` die gleichnamige Methode des Connectors und beendet die Verbindung mit der Datenbank.

```
public static void closeConnection() {
    LOG.trace("close SQL Server Connection...");
    Connector.closeConnection(sqlServerConnection);
}
```

Tabellen-Schema

Das Tabelle-Schema repräsentiert eine Relation der Datenbank. Hier dient es dazu, einen lesenden Zugriff auf die Datenbank programmatisch zu ermöglichen. Es besteht aus einer einzigen Methode `selectAllFromTable()`, die eine Multiselect-Query auf einer Tabelle der Datenbank realisiert. Erst an dieser Stelle wird natives SQL verwendet.

```
public static void selectAllFromTable() {
    Statement statement;
    ResultSet resultSet;
    String selection = "select id, ecokey from CBM.dbo.ECO";

    try {
        statement = sqlServerConnection.createStatement();
        resultSet = statement.executeQuery(selection);
        while(resultSet.next()) {
            String eco_id = resultSet.getString("ID");
            String eco_key = resultSet.getString("ECOKEY");
            LOG.trace(eco_id + ":" + eco_key);
        }
    } catch(SQLException sqlException) {
        System.err.println(sqlException.getMessage());
    }
}
```

Anwendungssteuerung

Die Klasse `Control` repräsentiert die Anwendungssteuerung der Anwendung und ist damit die wichtigste Klasse der Anwendung. Sie ist in diesem Fall eine leere Hülle, die eigentlich nur zu Delegationszwecken besteht, aber ein Bindeglied zur Datenbank und Datenverarbeitung darstellt. Dabei ist die rückgabelose und parameterfreie Methode `run()` die Hauptmethode der Anwendungssteuerung, die von der Hauptmethode der Hauptklasse gerufen wird. An dieser Stelle wird die fachliche und technische Logik der Anwendung gesteuert.

```
public static void run(String computerName) {
    LOG = LogManager.getLogger(Control.class.getName());
    password = initialize(computerName);
    start(password);
    process();
    complete();
}
```

Diese Methode ruft vier Subroutinen, um die Steuerung zu initialisieren, die Verarbeitung zu beginnen, durchzuführen und abzuschließen. Die Initialisierung wird von der Methode `initialize()` vorgenommen, die die Datenbank-Properties setzt und lädt sowie in Abhängigkeit von dem Rechnernamen das zugehörige Passwort setzt, welches gleichzeitig auch der Rückgabewert der Methode ist.

```

public static String initialize(String computerName) {

    // Database-Properties setzen und laden
    DatabaseProperty.setProperties();
    DatabaseProperty.load();

    // Werte laden
    DATABASE_USER = DatabaseProperty.getDatabaseUser();
    DATABASE_PASSWORD_TEST = DatabaseProperty.getDatabasePasswordTest();
    DATABASE_PASSWORD_META = DatabaseProperty.getDatabasePasswordMeta();
    DATABASE_SERVER_TEST = DatabaseProperty.getDatabaseServerTest();
    DATABASE_SERVER_META = DatabaseProperty.getDatabaseServerMeta();
    DATABASE_DRIVER = DatabaseProperty.getDatabaseDriver();
    DATABASE_PORT = DatabaseProperty.getDatabasePort();

    // Abfrage des Rechnernamens und setzen des Passworts
    if (DATABASE_SERVER_META.equals(computerName)) {
        password = DATABASE_PASSWORD_META;
    } else if (DATABASE_SERVER_TEST.equals(computerName)) {
        password = DATABASE_PASSWORD_TEST;
    } else {
        LOG.error("Keine Server definiert");
    }

    return password;
}

```

Anschließend ruft die Hauptroutine die Start-Methode start() mit dem Datenbank-Passwort als Argument. Die Start-Methode verwendet das Domänen-Schema und stellt eine Verbindung zur Datenbank her.

```

public static void start(String databasePassword) {
    LOG.trace("Starten der Anwendungssteuerung");
    Domain.getConnection(DATABASE_DRIVER, DATABASE_PORT, DATABASE_USER, databasePassword);
    LOG.trace("Verbinden mit der Datenbank");
}

```

Im nächsten Schritt wird die Anwendungssteuerung durch die Methode process() verarbeitet. Diese Methode ruft das Tabellen-Schema, um eine einfache Selektion auf der Datenbank auszuführen.

```

public static void process() {
    LOG.trace("Verarbeiten der Anwendungssteuerung");
    Table.selectAllFromTable();
    LOG.trace("Lesen der Datenbank");
}

```

Den Abschluss der Anwendungssteuerung bildet die Methode complete(), welche über das Domänen-Schema die Verbindung zur Datenbank wieder trennt.

```

public static void complete() {
    LOG.trace("Beenden der Anwendungssteuerung");
    Domain.closeConnection();
    LOG.trace("Trennen der Datenbank");
}

```